

Get Started: Install on macOS

- System requirements
- Get the Flutter SDK
 - Clone the repo
 - Run flutter doctor
 - Update your path
- Editor setup
- Platform setup
- iOS setup
 - Install Xcode
 - Set up the iOS simulator
 - Deploy to iOS devices
- Android setup
 - Install Android Studio
 - Set up your Android device
 - Set up the Android emulator
- Next step

System requirements

To install and run Flutter, your development environment must meet these minimum requirements:

- **Operating Systems:** macOS (64-bit)
- **Disk Space:** 700 MB (does not include disk space for Xcode or Android Studio).
- **Tools:** Flutter depends on these command-line tools being available in your environment.
 - `bash`, `mkdir`, `rm`, `git`, `curl`, `unzip`, `which`

Get the Flutter SDK

To get Flutter, use `git` to clone the repository and then add the `flutter` tool to your path. Running `flutter doctor` shows any remaining dependencies you may need to install.

Clone the repo

If this is the first time you're installing Flutter on this machine, clone the `beta` branch of the repository and then add the `flutter` tool to your path:

```
$ git clone -b beta https://github.com/flutter/flutter.git
$ export PATH=`pwd`/flutter/bin:$PATH
```

The above command sets your PATH variable temporarily, for the current terminal window. To permanently add Flutter to your path, see [Update your path](#).

To update an existing version of Flutter, see [Upgrading Flutter](#).

Run flutter doctor

Run the following command to see if there are any dependencies you need to install to complete the setup:

```
$ flutter doctor
```

This command checks your environment and displays a report to the terminal window. The Dart SDK is bundled with Flutter; it is not necessary to install Dart separately. Check the output carefully for other software you may need to install or further tasks to perform (shown in **bold text**).

For example:

```
[!] Android toolchain - develop for Android devices
    • Android SDK at /Users/obiwan/Library/Android/sdk
    x Android SDK is missing command line tools; download from https://goo.gl/XxQghQ
    • Try re-installing or updating your Android SDK,
      visit https://flutter.io/setup/#android-setup for detailed instructions.
```

La primera vez que ejecuta un comando flutter (como **flutter doctor**), descarga sus propias dependencias y compila sí mismo. Las ejecuciones posteriores deberían ser mucho más rápidas.

Las siguientes secciones describen cómo realizar estas tareas y finalizar el proceso de configuración. Ya lo verás **flutter doctor** salida que si elige utilizar un IDE, complementos están disponibles para IntelliJ IDEA, Android Studio y VS Code. Ver Configuración del editor para conocer los pasos para instalar los complementos Flutter y Dart.

Una vez que haya instalado las dependencias faltantes, ejecute el **flutter doctor** ordene nuevamente a verifica que hayas configurado todo correctamente.

El **flutter** La herramienta utiliza Google Analytics para informar de forma anónima las estadísticas de uso de funciones y informes básicos de accidentes. Estos datos se utilizan para ayudar a mejorar las herramientas de Flutter con el tiempo. Los análisis no se envían en la primera ejecución ni para ninguna ejecución que involucre **flutter config**, para que pueda optar por no realizar análisis antes de enviar cualquier dato. Para deshabilitar los informes, tipo **flutter config --no-analytics** y para mostrar la configuración actual, escriba **flutter config**. Consulte la política de privacidad de Google: www.google.com/intl/en/policies/privacy.

Actualiza tu ruta

Puede actualizar su variable PATH para la sesión actual solo en la línea de comando como se muestra en Clonar el repositorio de Flutter. Probablemente querrás hacerlo actualice esta variable permanentemente para que pueda ejecutarla **flutter** comandos en cualquier sesión de terminal.

Los pasos para modificar esta variable de forma permanente para todas las sesiones de terminal son específicos de la máquina. Normalmente agregas una línea a un archivo que se ejecuta cada vez que lo abres Una nueva ventana. Por ejemplo:

1. Determina el directorio donde colocaste el SDK de Flutter. Tú lo harás Necesito esto en el paso 3.
2. Abrir (o crear) `$HOME/.bash_profile` . La ruta y el nombre del archivo pueden ser diferente en tu máquina.
3. Añade la siguiente línea y cambia `[PATH_TO_FLUTTER_GIT_DIRECTORY]` ser La ruta donde clonaste el repositorio git de Flutter:

```
$ export PATH=[PATH_TO_FLUTTER_GIT_DIRECTORY]/flutter/bin:$PATH
```

1. Correr `source $HOME/.bash_profile` para actualizar la ventana actual.
2. Verifique que el `flutter/bin` El directorio ahora está en su PATH ejecutando:

```
$ echo $PATH
```

Para obtener más detalles, consulte Esta pregunta de StackExchange.

Configuración del editor

Usando el **flutter** Herramientas de línea de comandos, puedes usar cualquier editor para desarrollar aplicaciones Flutter. Tipo `flutter help` en un momento para ver las herramientas disponibles.

Recomendamos utilizar nuestros complementos para a Rica experiencia en IDE Admite edición, ejecución y depuración de aplicaciones Flutter. Ver Configuración del editor para pasos detallados.

Configuración de la plataforma

macOS admite el desarrollo de aplicaciones Flutter tanto para iOS como para Android. Completo en al menos uno de los dos pasos de configuración de la plataforma ahora, para poder construir y ejecutar su Primera aplicación Flutter.

Configuración de iOS

Instalar Xcode

Para desarrollar aplicaciones Flutter para iOS, necesitas una Mac con Xcode 7.2 o más reciente:

1. Instale Xcode 7.2 o más reciente (a través de descarga web o el Tienda de aplicaciones para Mac).
2. Configure las herramientas de línea de comandos de Xcode para utilizar la versión recién instalada de Xcode mediante corriendo `sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer` desde la línea de comando.

Esta es la ruta correcta para la mayoría de los casos, cuando desea utilizar la última versión de Xcode. Si necesita utilizar una versión diferente, especifique esa ruta.

3. Asegúrese de que el acuerdo de licencia de Xcode esté firmado abriendo Xcode una vez y confirmando o corriendo `sudo xcodebuild -license` desde la línea de comando.

Con Xcode, podrás ejecutar aplicaciones Flutter en un dispositivo iOS o en el simulador.

Configurar el simulador de iOS

Para prepararse para ejecutar y probar su aplicación Flutter en el simulador de iOS, siga estos pasos:

1. En tu Mac, busca el Simulador a través de Spotlight o usando el siguiente comando:

```
$ open -a Simulator
```

1. Asegúrese de que su simulador esté utilizando un dispositivo de 64 bits (iPhone 5s o posterior) verificando la configuración en el simulador **Hardware > Dispositivo** menú.
2. Dependiendo del tamaño de pantalla de su máquina de desarrollo, simule dispositivos iOS de alta densidad de pantalla puede desbordar su pantalla. Establezca la escala del dispositivo debajo de **Ventana > Escala** menú en el simulador.
3. Inicie su aplicación ejecutándola `flutter run`.

Implementar en dispositivos iOS

Para implementar tu aplicación Flutter en un dispositivo iOS físico, necesitarás algunas herramientas adicionales y una cuenta de Apple. También necesitarás configurar la implementación del dispositivo físico en Xcode.

1. Instalar homebrew.
2. Abra la terminal y ejecute estos comandos para instalar las herramientas para implementar aplicaciones Flutter Dispositivos iOS.

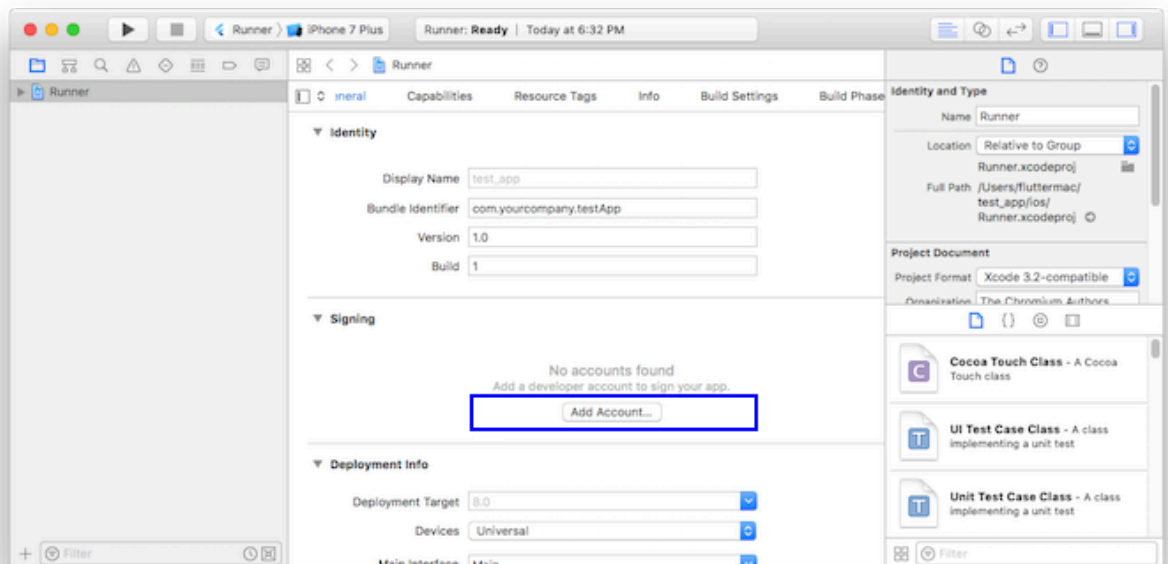
```
$ brew update  
$ brew install --HEAD libimobiledevice
```

```
$ brew install deviceinstaller ios-deploy cocoapods  
$ pod setup
```

Si alguno de estos comandos falla con un error, ejecute **brew doctor** y sigue las instrucciones para resolver el problema.

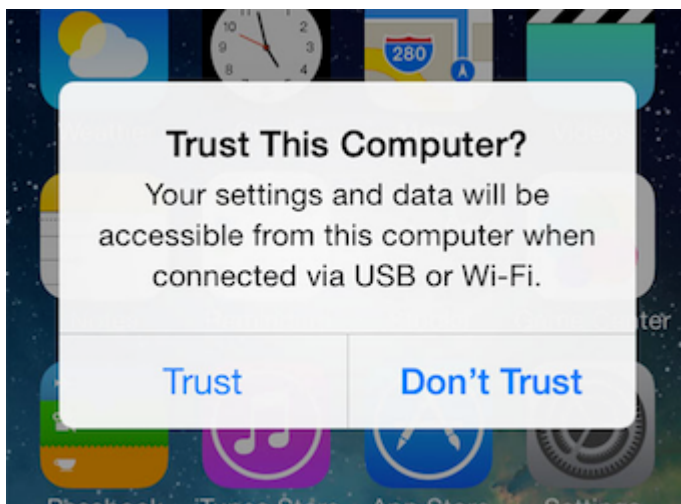
1. Siga el flujo de firma de Xcode para aprovisionar su proyecto:

- Abra el espacio de trabajo predeterminado de Xcode en su proyecto ejecutando **open ios/Runner.xcworkspace** en una ventana de terminal desde el directorio de su proyecto Flutter.
- En Xcode, seleccione el **Runner** proyecto en el panel de navegación izquierdo.
- En el **Runner** Página de configuración de destino, asegúrese de que su equipo de desarrollo esté seleccionado en **General > Firma > Equipo**. Cuando selecciona un equipo, Xcode crea y descarga un Certificado de desarrollo, registra su dispositivo con su cuenta y crea y descarga un perfil de aprovisionamiento (si es necesario).
 - Para comenzar su primer proyecto de desarrollo de iOS, es posible que deba iniciar sesión en Xcode con su ID de Apple.



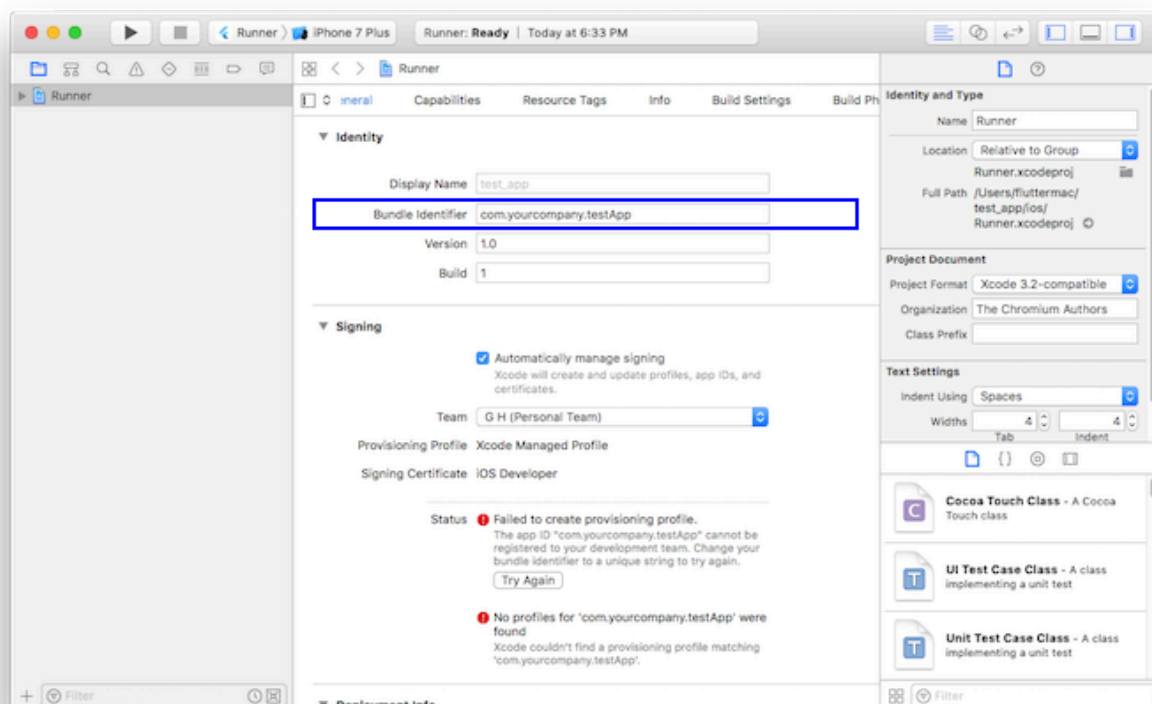
El desarrollo y las pruebas son compatibles con cualquier ID de Apple. Es necesario inscribirse en el Programa para desarrolladores de Apple para distribuir su aplicación a la App Store. Ver el Diferencias entre los tipos de membresía de Apple.

- La primera vez que utilice un dispositivo físico adjunto para el desarrollo de iOS, deberá confiar tanto en su Mac como en el Certificado de desarrollo de ese dispositivo. Seleccionar **Trust** en el mensaje de diálogo cuando conecte por primera vez el dispositivo iOS a su Mac.



Luego, vaya a la aplicación Configuración en el dispositivo iOS y seleccione **General > Gestión de dispositivos** y confíe en su Certificado.

- Si falla la firma automática en Xcode, verifique que el proyecto **General > Identidad > Identificador de paquete** El valor es único.



2. Inicie su aplicación ejecutándola `flutter run`.

Configuración de Android

Nota: Flutter se basa en una instalación completa de Android Studio para suministrar sus dependencias de la plataforma Android. Sin embargo, puedes escribir tu Aplicaciones Flutter en varios editores; un paso posterior discutirá eso.

Instalar Android Studio

1. Descargar e instalar Estudio Android.
2. Inicie Android Studio y pase por el 'Asistente de configuración de Android Studio'. Esto instalará el último SDK de Android, Android SDK Platform-Tools y Android SDK Herramientas de compilación, que Flutter requiere al desarrollar para Android.

Configura tu dispositivo Android

Para prepararse para ejecutar y probar su aplicación Flutter en un dispositivo Android, necesitará un Dispositivo Android que ejecuta Android 4.1 (nivel API 16) o superior.

1. Enable **Opciones para desarrolladores** y **Depuración USB** en tu dispositivo. Instrucciones detalladas están disponibles en el Documentación de Android.
2. Usando un cable USB, conecte su teléfono a su computadora. Si se le solicita en su dispositivo, autoriza a tu computadora a acceder a tu dispositivo.
3. En la terminal, ejecute el **flutter devices** comando para verificar que Flutter reconoce tu dispositivo Android conectado.
4. Inicie su aplicación ejecutándola **flutter run**.

De forma predeterminada, Flutter utiliza la versión del SDK de Android donde **adb** La herramienta está basada. Si Si desea que Flutter utilice una instalación diferente del SDK de Android, debe configurar el **ANDROID_HOME** variable de entorno a ese directorio de instalación.

Configurar el emulador de Android

Para prepararse para ejecutar y probar su aplicación Flutter en el emulador de Android, siga estos pasos:

1. Enable Aceleración de VM en tu máquina.
2. Lanzamiento **Android Studio>Herramientas>Android>Administrador AVD** y seleccione **Crear dispositivo virtual**.
3. Elija una definición de dispositivo y seleccione **Siguiente**.
4. Seleccione una o más imágenes del sistema para las versiones de Android que desea emular y seleccione **Siguiente**. An **x86** o **x86_64** Se recomienda imagen.
5. En Rendimiento emulado, seleccione **Hardware - GLES 2.0** para permitir aceleración de hardware.
6. Verifique que la configuración de AVD sea correcta y seleccione **Terminar**.

Para obtener detalles sobre los pasos anteriores, consulte Gestión de AVD.

7. En Android Virtual Device Manager, haga clic **Correr** en la barra de herramientas. El emulador se inicia y muestra el lienzo predeterminado para la versión del sistema operativo seleccionada y dispositivo.
8. Inicie su aplicación ejecutándola **flutter run**. El nombre del dispositivo conectado es **Android SDK built for <platform>**, donde **plataforma** es la familia de chips, como el x86.

Siguiente paso

Siguiente paso: Configurar el editor